DATA ENGINEERING MASTER PREP • EYE-CARE WARM PARCHMENT EDITION

Pillar 3: Lakehouse (Iceberg/Delta), Streaming & Orchestration — First-Principles Guide

Stream Processing, Watermarks & Stateful Windows • Data Lakehouse (Iceberg/Delta), Medallion & Orchestration

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

7 Sections Indexed

Day 5: Stream Processing, Watermarks & Stateful Windows ..... p. 2

1. Event Time vs. Processing Time & Watermarks (DE13) ..... p. 2
2. Streaming Window Geometries (DE14) ..... p. 2

Day 6: Data Lakehouse (Iceberg/Delta), Medallion & Orchestration ..... p. 3

1. Open Table Formats: Apache Iceberg & Delta Lake (DE15) ..... p. 3
2. Small File Compaction, Partitioning & Z-Ordering (DE16) ..... p. 4
3. Airflow Idempotency & The Medallion + WAP Architecture (DE17 , DE18) ..... p. 4

Day 5: Stream Processing, Watermarks & Stateful Windows

↑ Mục Lục

1. Event Time vs. Processing Time & Watermarks (`DE13`)

In distributed stream processing (Apache Flink, Spark Structured Streaming), every event carries two distinct timestamps:

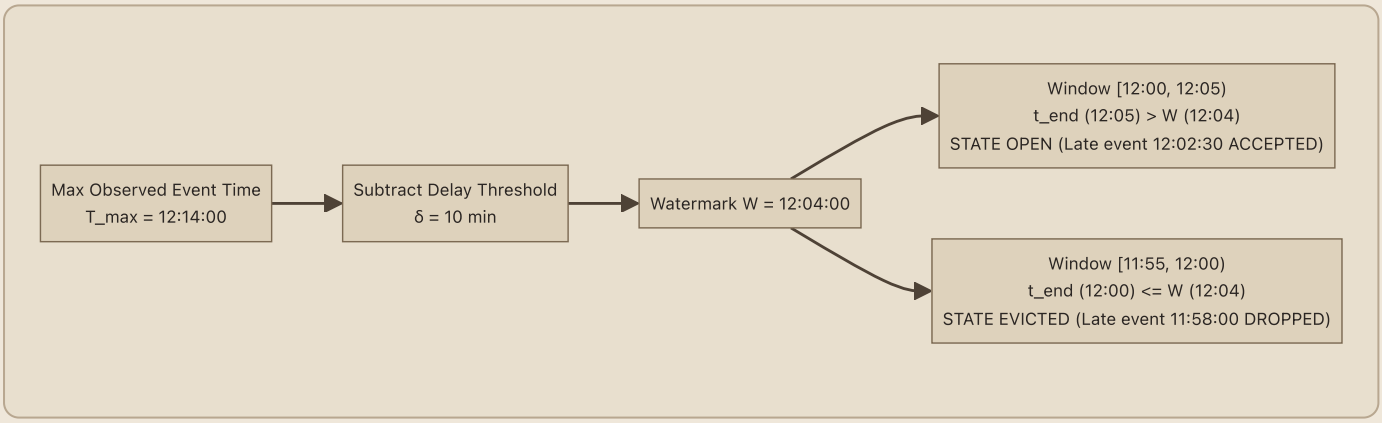
- **Event Time (t_{event}):** When the action physically occurred on the user's device (embedded inside the payload).
- **Processing Time (t_{proc}):** Wall-clock time of the server executing the streaming operator when the record is processed.

Why Watermarks Are Required for Event-Time Windows

Because mobile networks, retries, and Kafka partition lags delay records, events arrive **out of order**. If a streaming job computes 5-minute window aggregations `[12:00, 12:05)` by **Event Time**, how long should the engine keep that window's state in RocksDB/memory waiting for stragglers?

A **Watermark** $W(t)$ is a monotonically advancing timestamp threshold asserting that the stream is *complete* up to time $W(t)$:

$$W(t) = \max_{e \text{ seen so far}} \left(t_{\text{event}}(e) \right) - \delta_{\text{delay}}$$



- **Window Lifecycle Rule:** A window $[t_{\text{start}}, t_{\text{end}})$ stays open in state as long as $W < t_{\text{end}}$. As soon as $W \geq t_{\text{end}}$, the window emits its final result and its state is evicted (unless an explicit `allowedLateness` / side-output policy is configured).

2. Streaming Window Geometries (`DE14`)

Window Type	Geometry & Parameters	Windows per Event	Typical Engineering Use Case
Tumbling Window	Fixed size S, non-overlapping (`size = 5 min`).	**Exactly** 1	Minute/hourly revenue rollups (`[12:00, 12:05)`, `[12:05, 12:10)`).

Window Type	Geometry & Parameters	Windows per Event	Typical Engineering Use Case
Sliding (Hopping) Window	Fixed size S, advances every slide $L < S$ (`size = 10 min, slide = 2 min`).	$\frac{S}{L}$ (e.g., $10/2 = 5$)	Moving averages (e.g., 10-minute rolling error rate updated every 2 minutes).
Session Window	Dynamic variable length per key; closes after inactivity gap Δ_{gap} (`gap = 30 min`).	1 **merged session**	User browsing sessions, clickstream engagement funnels.

Day 6: Data Lakehouse (Iceberg/Delta), Medallion & Orchestration

↑ Mục Lục

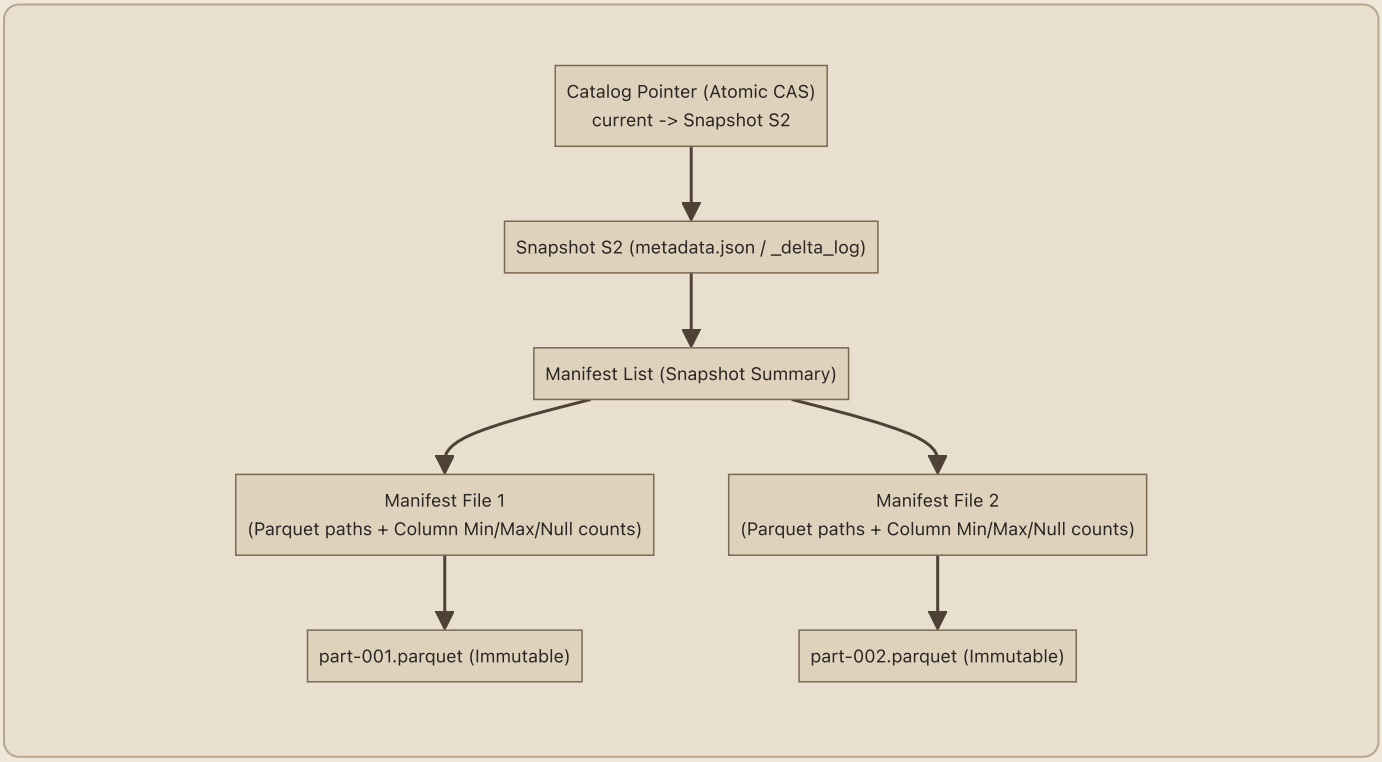
1. Open Table Formats: Apache Iceberg & Delta Lake (`DE15`)

Classic Hive-style Data Lakes on object storage (`s3://bucket/table/dt=2026-03-01/*.parquet`) suffer from three fatal flaws:

1. **No Atomic Multi-File Commits:** If a Spark job crashes after writing 40 out of 100 Parquet files, readers see corrupted partial data.
2. **Slow Directory Listing (`S3 LIST`):** Planning a query requires listing thousands of prefixes over HTTP.
3. **No Safe Concurrent Reads & Writes:** Mutating or compacting files while readers scan them causes `FileNotFoundException` .

How Iceberg & Delta Lake Deliver ACID on Immutable Parquet

Instead of using directory listings as the source of truth, **Apache Iceberg** and **Delta Lake** maintain a versioned **Metadata Tree / Transaction Log** above immutable Parquet files:



- **Snapshot Isolation & Time Travel:** Readers read a pinned snapshot (S_n) and never see uncommitted files. Querying `SELECT * FROM events VERSION AS OF 42` or `TIMESTAMP AS OF '2026-03-01 00:00:00'` simply reads an older snapshot pointer before `VACUUM` / `expire_snapshots` deletes unreferenced files.
- **Optimistic Concurrency Control (OCC):** Concurrent writers prepare Parquet files optimistically and attempt an atomic compare-and-swap on the catalog/log pointer. Non-conflicting partition writes automatically rebase and succeed; conflicting row-level writes retry or abort cleanly.

---

2. Small File Compaction, Partitioning & Z-Ordering (`DE16`)

1. **The Small File Problem & Compaction (`OPTIMIZE` / `rewrite_data_files`):**
   - High-frequency streaming sinks write thousands of 1 MB files per day, degrading query latency due to S3 metadata and HTTP connection overhead.
   - **Bin-Packing Compaction** periodically rewrites small files into optimal 128 MB–1 GB Parquet files in a new snapshot.
2. **Coarse Physical Partitioning vs. Multi-Dimensional Z-Ordering:**
   - **Physical Partitioning (`PARTITIONED BY (event_date)`):** Only partition by **low-cardinality** columns ($\leq 10^3$–10^4 partitions total) so each partition holds ≥ 1 GB. Partitioning by high-cardinality columns like `user_id` causes catastrophic **over-partitioning** (billions of tiny files).
   - **Why Linear `ORDER BY (colA, colB)` Fails on `colB`:** Linear sorting clusters `colA` tightly, but spreads `colB` across every file, resulting in 0% file skipping for queries filtering only on `colB`.
   - **Z-Ordering (`ZORDER BY (user_id, merchant_id)`):** Interleaves the bits of multiple columns along a **Morton space-filling curve**, preserving locality in 2D/3D space so Parquet file `[min, max]` statistics skip $> 90\%$ of files whether filtering on `user_id`, `merchant_id`, or both!

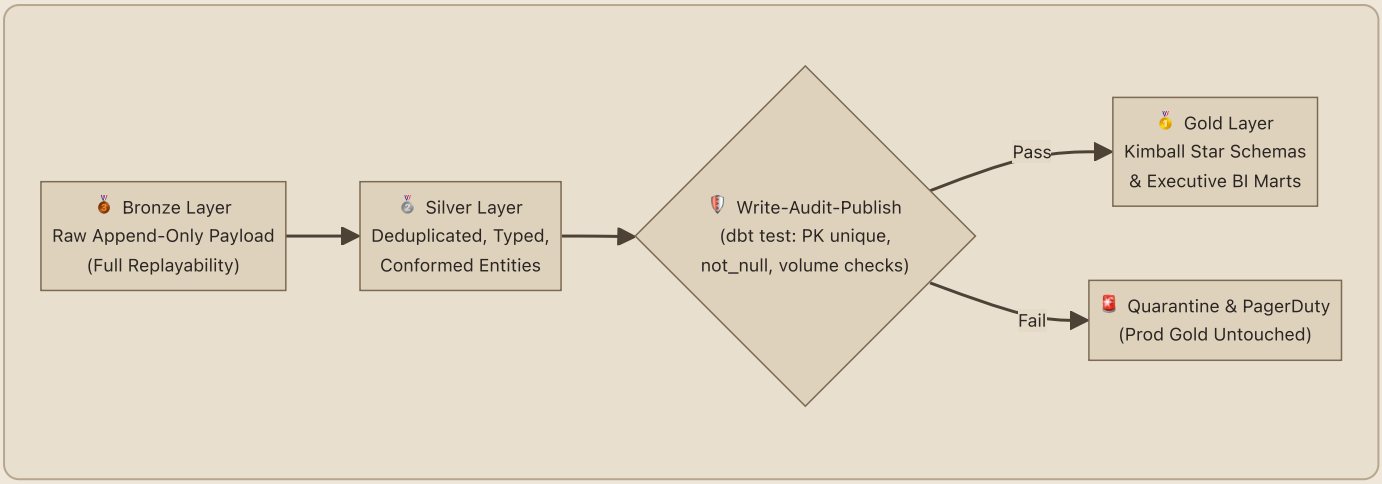
---

3. Airflow Idempotency & The Medallion + WAP Architecture (`DE17`, `DE18`)

Two Golden Rules of Idempotent Airflow DAGs (`DE17`)

1. **Deterministic Time Boundaries (`data_interval_start` / `{{ ds }}`):** Never use `NOW()` or `CURRENT_DATE()` inside SQL tasks. Always filter using Airflow's logical interval (`WHERE ts >= '{{ data_interval_start }}' AND ts < '{{ data_interval_end }}'`) so retries and historical **backfills** (`airflow dags backfill`) process the exact target window.
2. **Atomic Partition Overwrite (`INSERT OVERWRITE` / `MERGE`):** Never use blind `INSERT INTO` on retryable tasks. Overwrite only the target logical date partition (`INSERT OVERWRITE TABLE fct PARTITION (dt = '{{ ds }}')`) so running the task $1\times$ or $10\times$ yields the exact same row count.

Medallion Architecture & Write-Audit-Publish (`DE18`)



- **Bronze:** Raw, immutable, append-only source payloads + ingestion metadata (`_ingested_at`). Enables full historical replay if downstream parsing logic changes.
- **Silver:** Deduplicated (`ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) = 1`), schema-enforced, conformed core entities.
- **Gold:** Business-ready Star Schemas (`fct_*`, `dim_*`), ML feature sets, and pre-aggregated BI marts.
- **Write-Audit-Publish (WAP):** Writes transformed records to an isolated staging table or catalog branch (`Write`), executes automated data quality contracts (`Audit` via `dbt test` / Great Expectations), and atomically swaps/merges into production (`Publish`) only if 100% of checks pass.